SOFTWARE ENGINEERING PROJECT GROUP 4

Deliverable 1: Project Title/Planning/Requirements Analysis

Title: CampusSeat

Description

CampusSeat is a web-based information system that allows students, staff and other people with campus access to discover university events and reserve a seat online. Administrators create, configure and delete events, setting details such as date, venue and seating capacity, and can choose to open an event to everyone or restrict it to specific schools.

Registered users browse the available events and apply for a seat by entering their credentials, which are validated against the event's eligibility rules and saved in the database. If seats remain and the user is eligible, the booking is confirmed automatically and the system issues a registration card as proof of attendance.

The system tracks available seats in real time so an event is never overbooked, and it gives administrators reports on bookings and attendance. It replaces manual sign-up sheets and scattered emails with one organised, accessible platform.

Group Members & Task Allocation

Member	Main responsibility	Tasks
1. **Edenmike Mwaura Ruhanga:** Frontend	User interface for both roles	User pages (browse events, booking form, registration card page, login/register) and admin panel pages (event management, bookings view, user management)
2. **Uthman Salyani:** Backend	Logic, database & security	DB creation, booking transaction (seat check & school eligibility), login & access levels, password

		hashing, PDF card generation, reports, audit logs
3. **Rick Kyalo:** Documentation, diagrams & mockups	Design & paperwork	Flow chart, use cases, class & ER diagrams, database schema, UI mockups, user manual, testing documentation

Functional Requirements

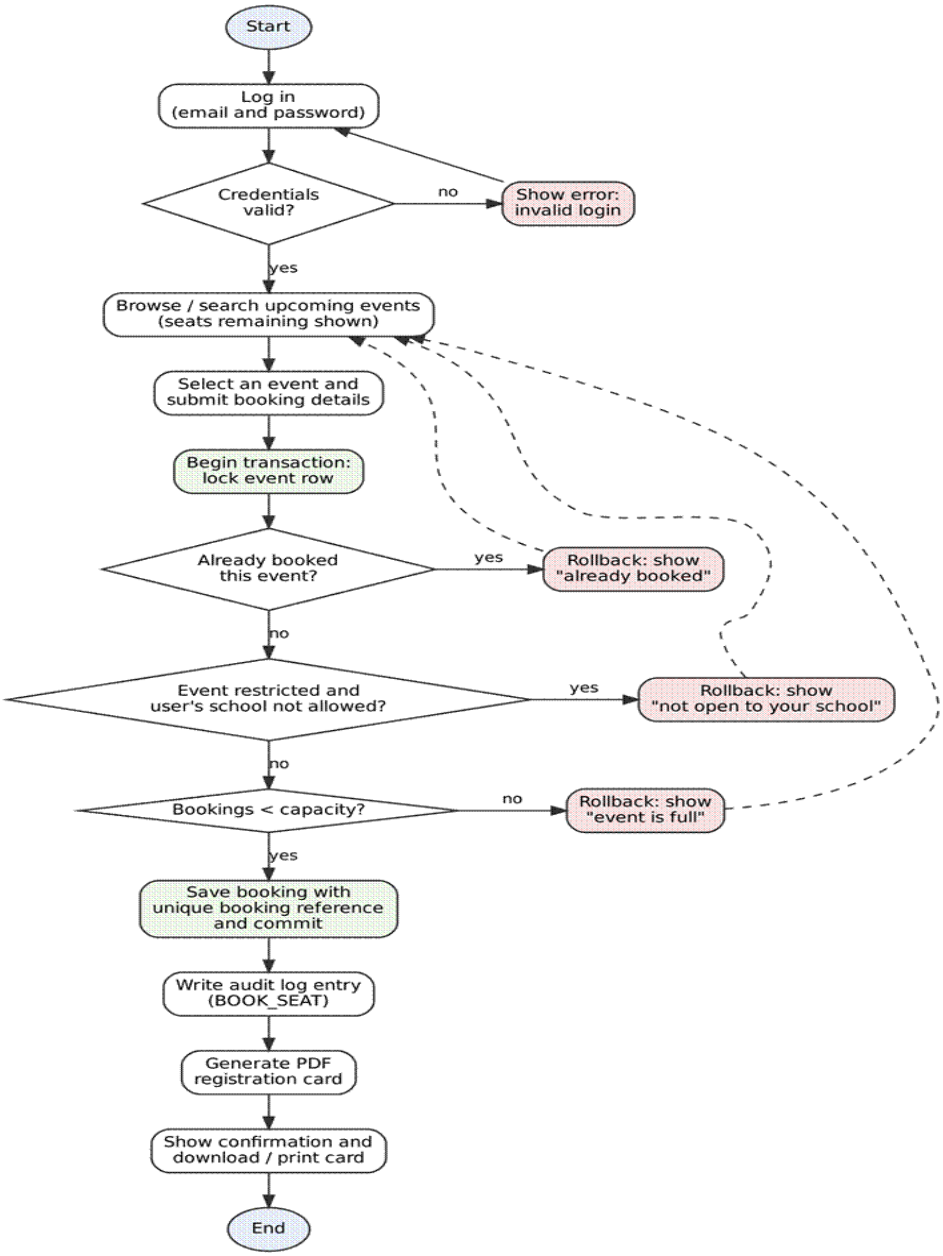
#	Requirement	Who it serves
FR1	The system shall allow users to register, log in and log out using a secure account, and shall restrict access to features based on role (user or admin)	**Users, admins**
FR2	The system shall allow admins to create, edit and delete events, including title, description, date, time, venue, seating capacity and an optional school restriction.	**Admins**
FR3	The system shall allow users to browse and search upcoming events and view the seats remaining for each.	**Users**
FR4	The system shall allow an eligible user to book a seat by submitting their credentials (name, school, student number, etc.), automatically confirm the booking if seats remain, reduce the available seats, and prevent duplicate bookings for the same event	**Users**
FR5	The system shall generate a downloadable, printable PDF registration card for each confirmed booking, showing the user's name, event, date, venue and a unique booking reference	**Users**

Non-Functional Requirements

#	Category	Requirement
NFR1	Security	The system shall store passwords as salted hashes (never plain text) and shall use parameterized queries for all database access to prevent SQL injection. Admin pages shall be accessible only to accounts with the admin role.
NFR2	Usability	The system shall have a consistent, responsive interface that works on both desktop and mobile browsers, and shall display clear error messages for invalid input (for example, a full event or a school restriction).
NFR3	Reliability & data integrity	The system shall never allow an event to be booked beyond its seating capacity, even when several users book at the same time, and shall load main pages within 3 seconds under normal use.

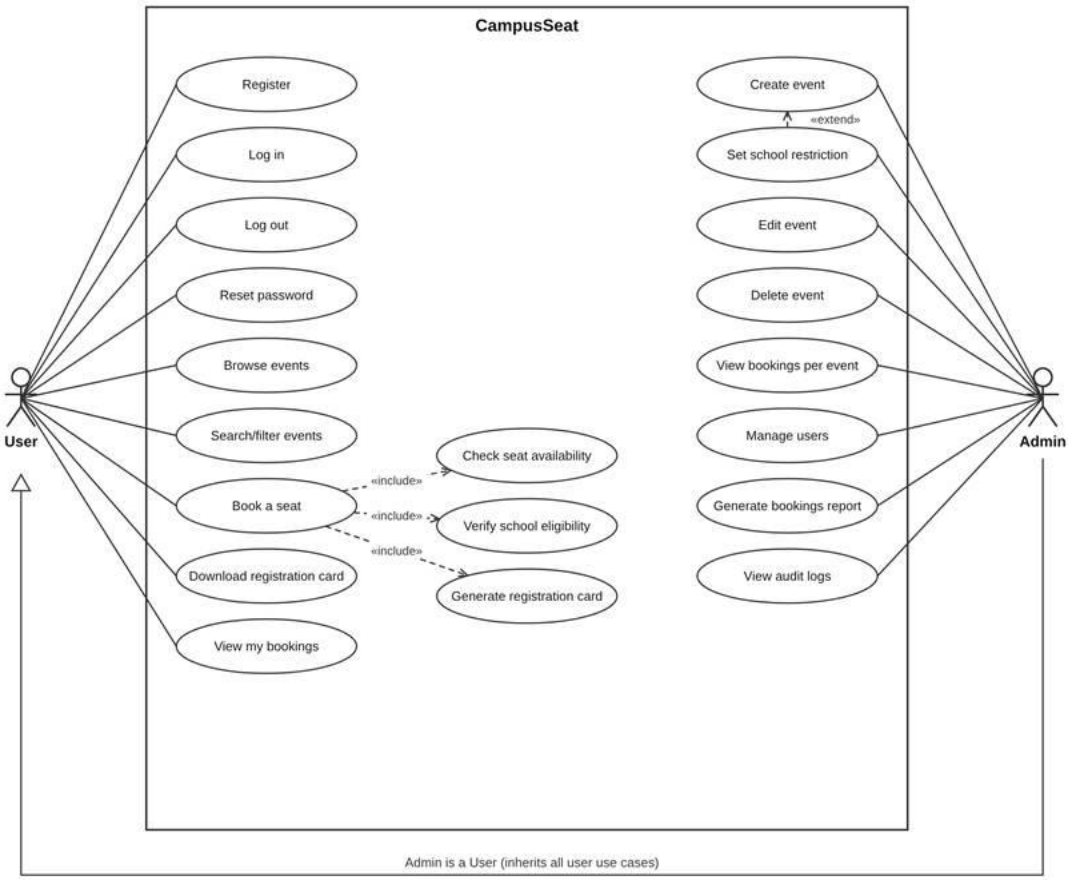
Deliverable 2: Diagrams, Database Schema and Mockups

1. Flow chart: booking a seat



A user logs in, picks an event and submits their details. The server then checks, inside one locked transaction, for a duplicate booking, school eligibility and free seats. Any failed check rolls back and shows an error; only a passing booking is saved, logged and turned into a PDF registration card.

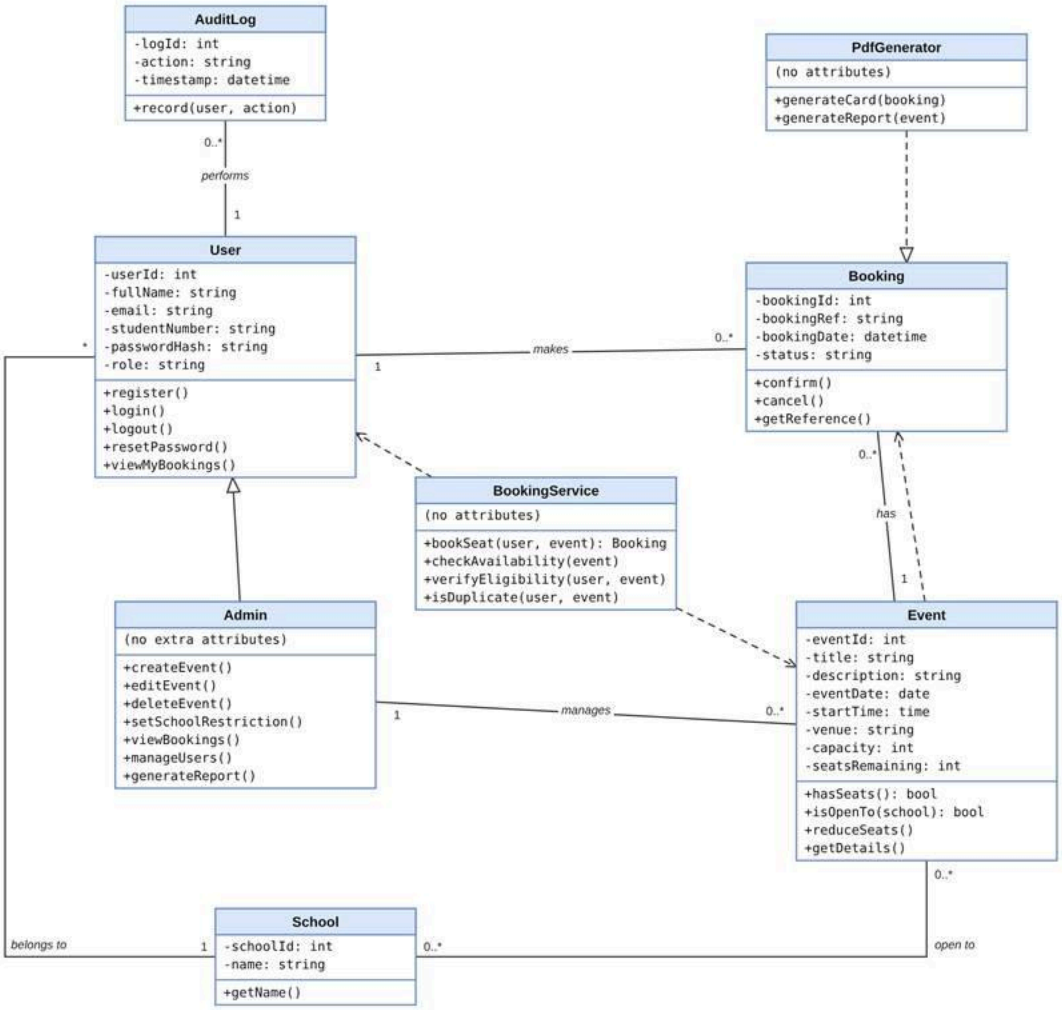
2. Use case diagram



Two actors use the system. The User registers, logs in and out, resets a password, browses and searches events, books a seat, downloads the registration card and views their bookings. The Admin is also a User and additionally creates, edits and deletes events, sets school restrictions, views bookings per event, manages users, generates bookings reports and views audit logs.

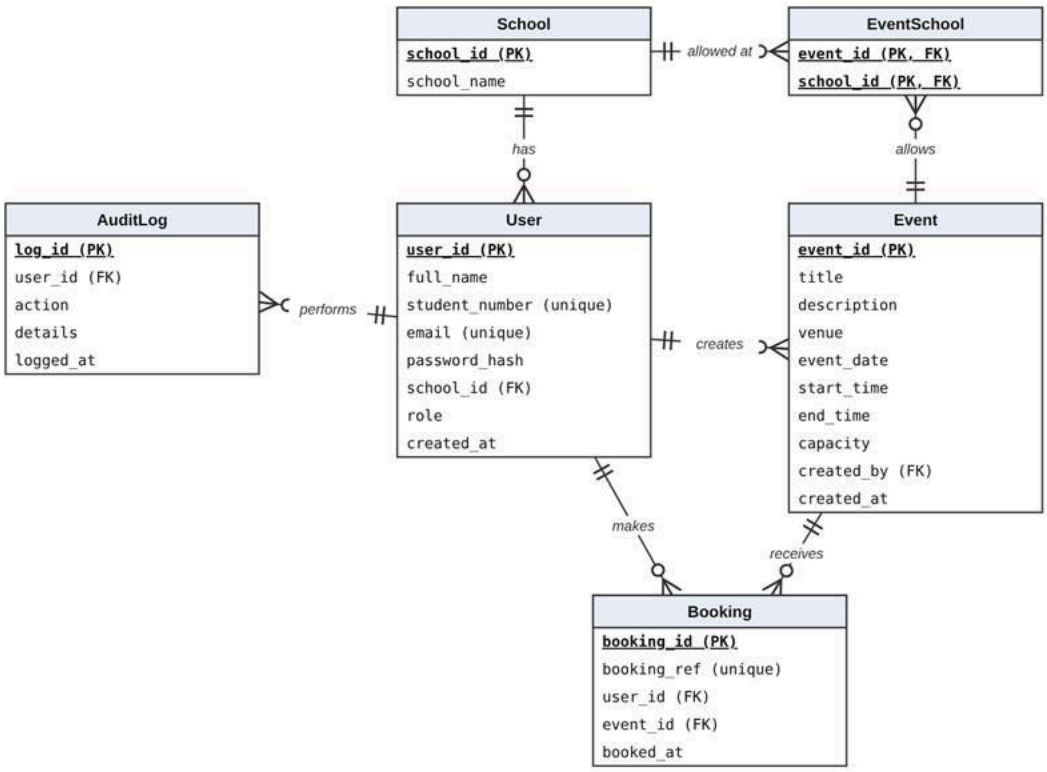
Booking a seat *includes* three steps: check seat availability, verify school eligibility and generate the registration card. Setting a school restriction *extends* creating an event, because it is optional.

3. Class diagram



The diagram shows User, Admin, Event, Booking, School and AuditLog, which hold the data, plus two service classes. BookingService runs the booking checks (availability, school eligibility, duplicates) and PdfGenerator produces the registration card and reports. Admin inherits from User, because the **role** column decides which methods a signed-in person may use. Events and schools are many-to-many ("open to"); an event with no school restriction is open to everyone.

4. Entity-relationship (ER) diagram



Relationship	Cardinality	Meaning
School has User	1 to many	Each user belongs to exactly one school.
User makes Booking	1 to many	A user can hold many bookings, one per event.
Event receives Booking	1 to many	An event takes bookings up to its capacity.
User creates Event	1 to many	Each event is created by one admin.
Event allows EventSchool	1 to many	An event lists the schools it is restricted to.
School allowed at EventSchool	1 to many	A school can be allowed at many events.

User performs AuditLog	1 to many	Each log entry records the user who acted.

5. Database schema

Database name: campusseat_db

DBMS: MySQL (InnoDB, so foreign keys are enforced) ·

Character set: utf8mb4. Table names are plural lowercase, columns use lowercase with underscores, and primary keys are <singular>_id.

schools: the schools or faculties used for profiles and event restrictions.

Column	Type	Constraints
school_id	INT	PK, AUTO_INCREMENT
school_name	VARCHAR(100)	NOT NULL, UNIQUE

users: every account (students, staff and admins).

Column	Type	Constraints
user_id	INT	PK, AUTO_INCREMENT
full_name	VARCHAR(100)	NOT NULL
student_number	VARCHAR(30)	NOT NULL, UNIQUE
email	VARCHAR(100)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL (hashed, never plain text)
school_id	INT	NOT NULL, FK to schools(school_id)
role	ENUM('user','admin')	NOT NULL, DEFAULT 'user'
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

events: the events that can be booked.

Column	Type	Constraints
event_id	INT	PK, AUTO_INCREMENT
title	VARCHAR(150)	NOT NULL
description	TEXT	NULL allowed
venue	VARCHAR(150)	NOT NULL
event_date	DATE	NOT NULL
start_time	TIME	NOT NULL
end_time	TIME	NOT NULL, CHECK (end_time > start_time)
capacity	INT	NOT NULL, CHECK (capacity > 0)
created_by	INT	NOT NULL, FK to users(user_id)
created_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

event_schools: links events to the schools allowed to book them. An event with no rows here is open to everyone.

Column	Type	Constraints
event_id	INT	PK (composite), FK to events(event_id), ON DELETE CASCADE
school_id	INT	PK (composite), FK to schools(school_id), ON DELETE CASCADE

bookings: each confirmed seat, the main transaction table.

Column	Type	Constraints
booking_id	INT	PK, AUTO_INCREMENT
booking_ref	VARCHAR(20)	NOT NULL, UNIQUE (e.g. CS-2026-00045)
user_id	INT	NOT NULL, FK to users(user_id)
event_id	INT	NOT NULL, FK to events(event_id), ON DELETE CASCADE
booked_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Table constraint: UNIQUE (user_id, event_id), so a user cannot book the same event twice.

audit_logs: important actions and who performed them.

Column	Type	Constraints
log_id	INT	PK, AUTO_INCREMENT
user_id	INT	NULL allowed, FK to users(user_id), ON DELETE SET NULL
action	VARCHAR(100)	NOT NULL (e.g. LOGIN, BOOK_SEAT, CREATE_EVENT)
details	TEXT	NULL allowed
logged_at	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Referential actions

Parent	Child	Foreign key	On delete
schools	users	users.school_id	**RESTRICT**
users	events	events.created_by	**RESTRICT**
users	bookings	bookings.user_id	**RESTRICT**
events	bookings	bookings.event_id	**CASCADE**
events	event_schools	event_schools.event_id	**CASCADE**
schools	event_schools	event_schools.school_id	**CASCADE**
users	audit_logs	audit_logs.user_id	**SET NULL**

Business rules enforced in the backend, inside one transaction

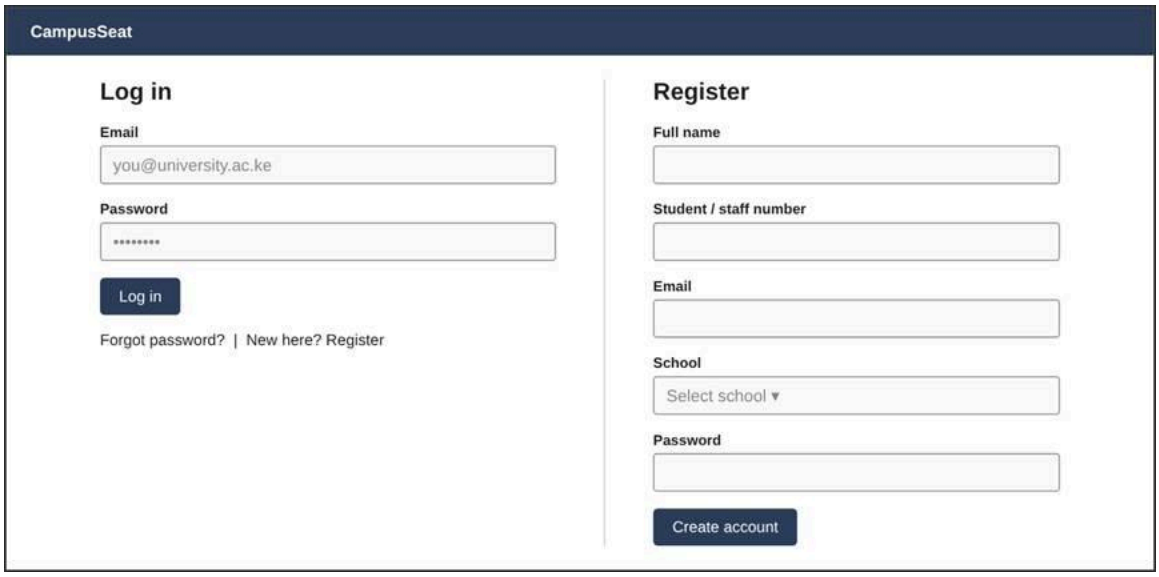
1. **Seat check:** book only if existing bookings for the event are fewer than capacity.
2. **Eligibility check:** if the event has event_schools rows, the user's school_id must be among them.
3. **No duplicates:** the unique (user_id, event_id) constraint rejects a second booking.
4. **No overbooking under simultaneous requests:** lock the event row (SELECT ... FOR UPDATE) before counting seats.

Seats remaining are calculated, not stored: capacity minus the count of that event's bookings.

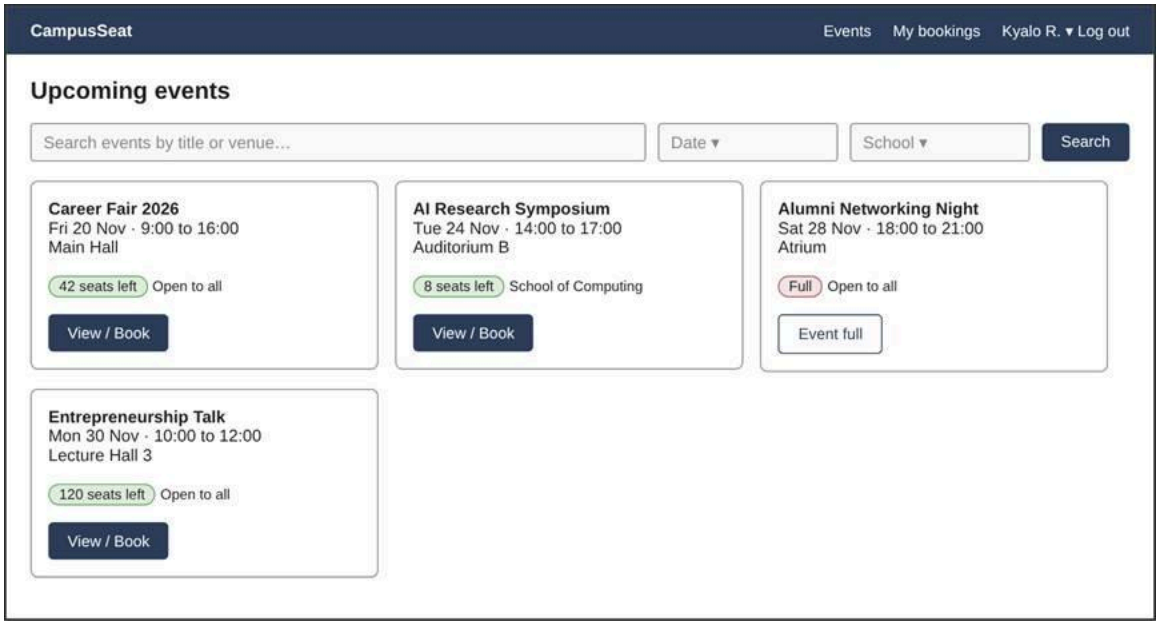
6. User interface mockups

Wireframes for the front-end (screens 1 to 4) and the admin panel (screens 5 to 9). Final styling may change during development.

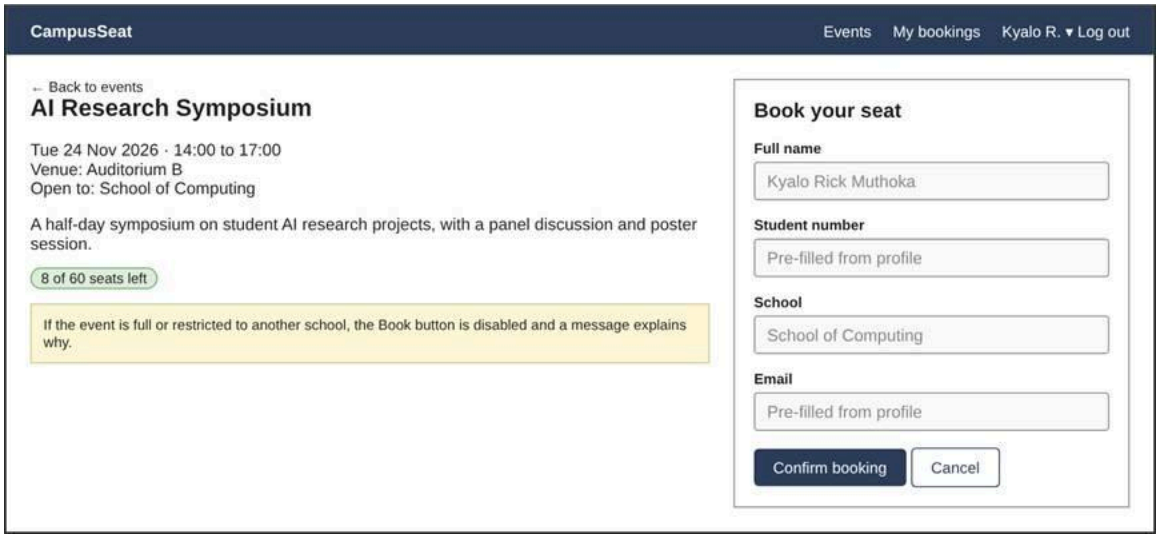
Front-end (users)



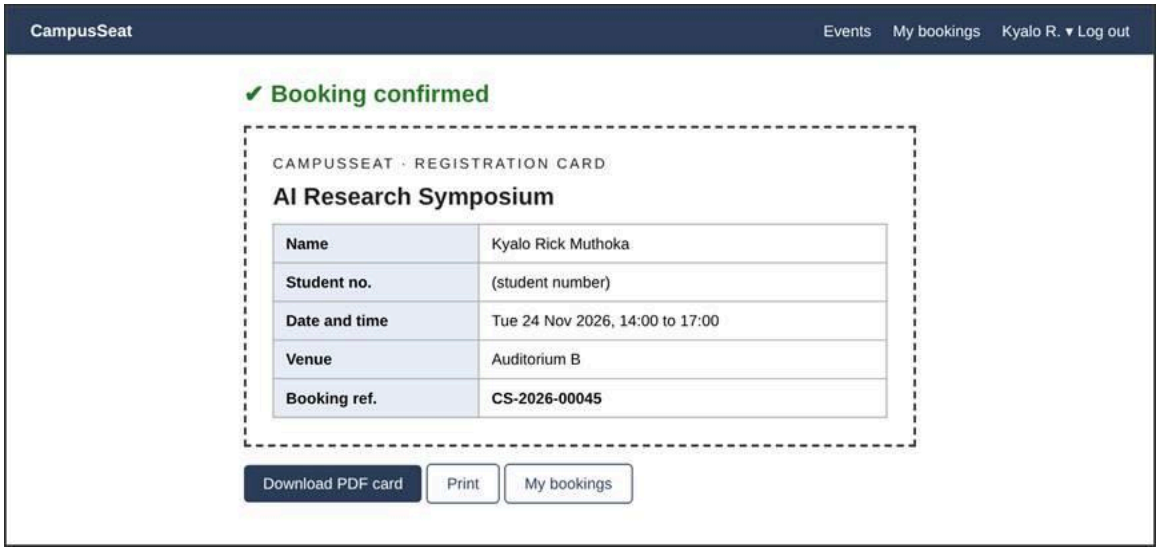
Screen 1: Login and registration (FR1)



Screen 2: Browse and search events, with seats remaining (FR3)

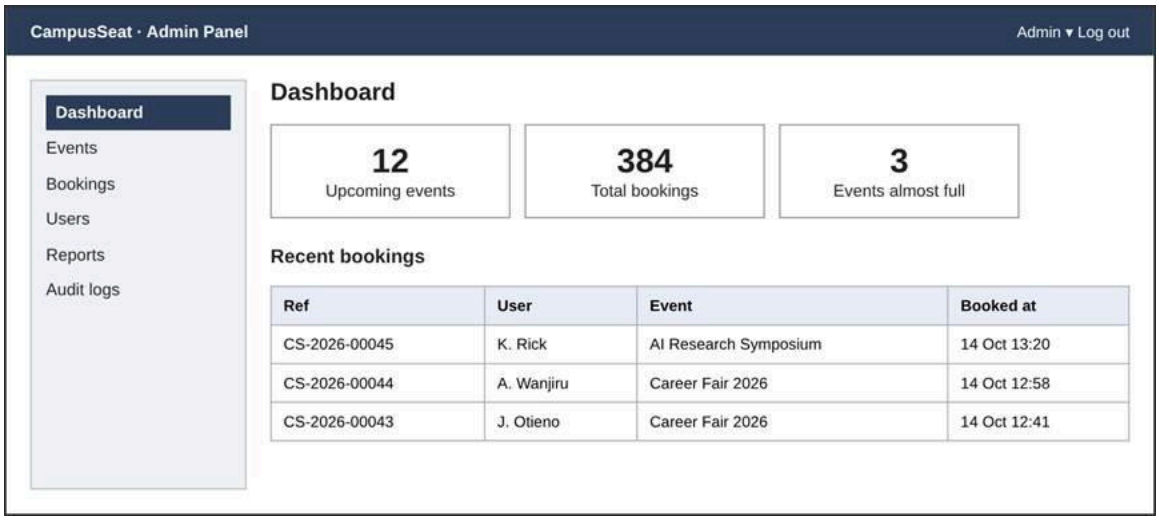


Screen 3: Event details and booking form (FR4)

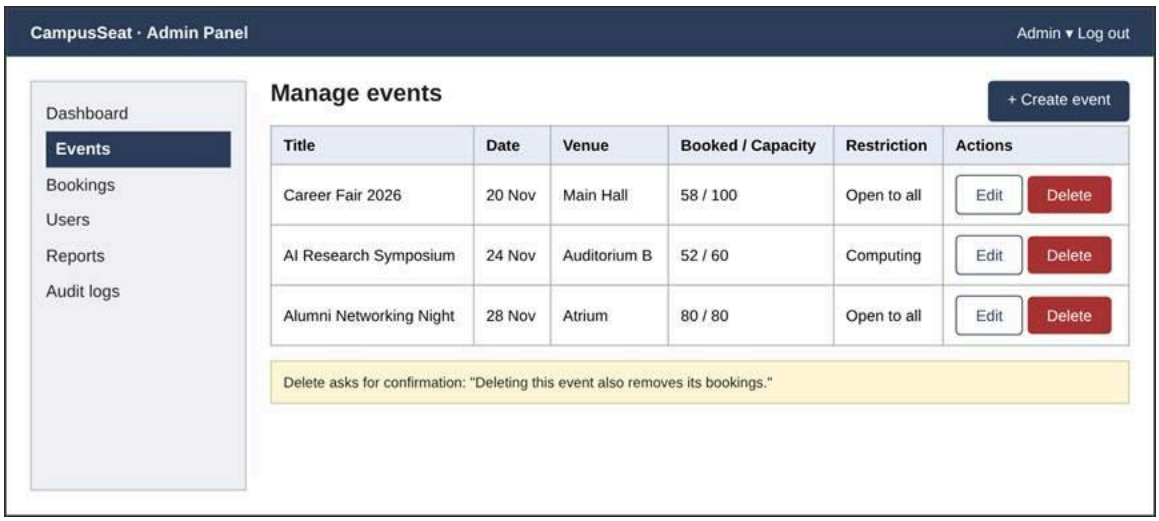


Screen 4: Booking confirmation and registration card (FR5)

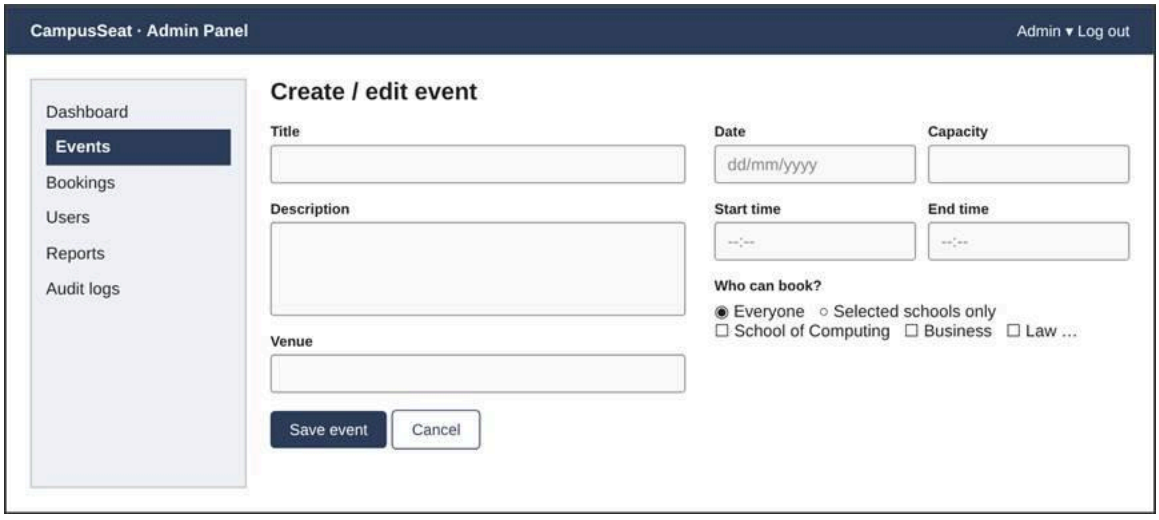
Admin panel



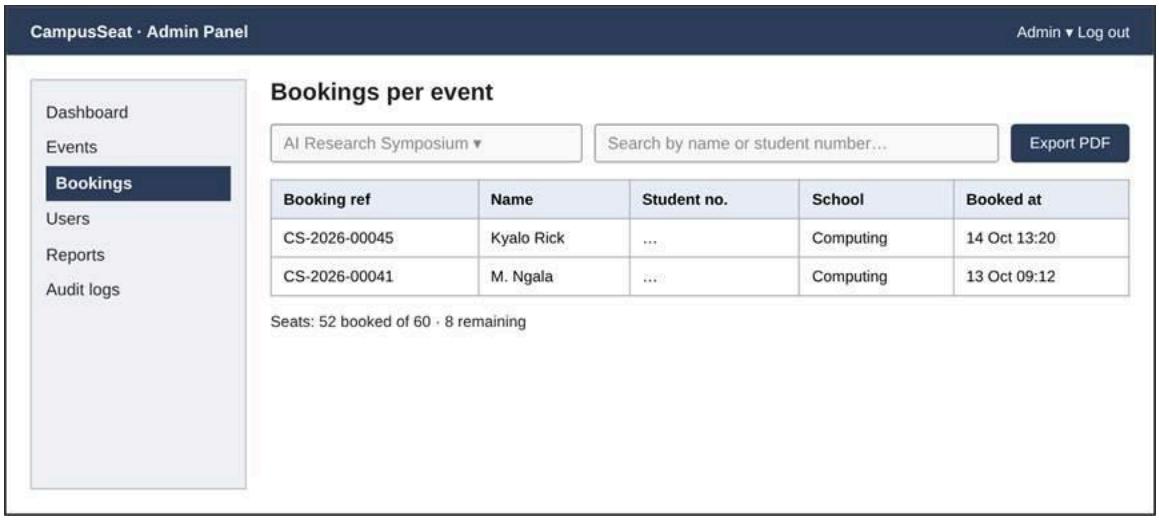
Screen 5: Admin dashboard



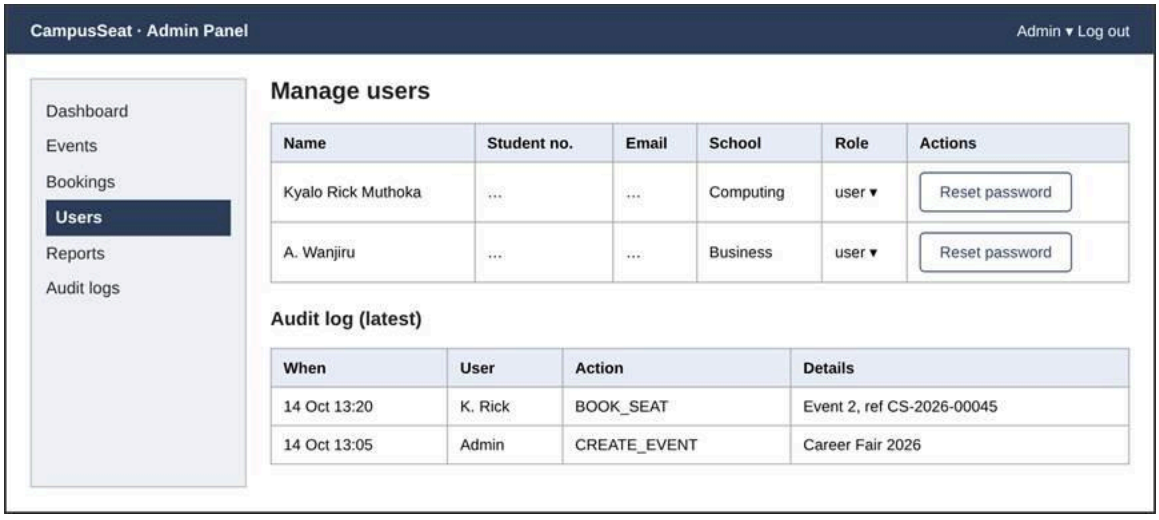
Screen 6: Event management list (FR2)



Screen 7: Create or edit event, with optional school restriction (FR2)



Screen 8: Bookings per event, with search and report export



Screen 9: User management and audit logs

Deliverable 3: progress checkpoints

All five mid-semester checkpoints are complete. The whole system, not only a prototype, is running.

Checkpoint	Status	Evidence to show
Database created	Done	MySQL database campusseat_db with 6 tables (schools, users, events, event_schools, bookings, audit_logs), foreign keys and indexes, created from database/schema.sql. Show it in MySQL Workbench.
Front-end created	Done	React 19 + Vite site: events, event detail and booking form, registration card, My bookings, login, register, forgot password, help, light and dark mode, mobile layout. See User Manual figures 1 to 15.
Admin panel created	Done	Dashboard, events, bookings, users, reports and audit logs behind an admin role. See User Manual figures 16 to 24.
Front-end connected to database	Done	The site calls a Node and Express API that reads and writes MySQL with prepared statements. Register a student, book a seat,

		then show the new rows in the bookings table.
Admin panel connected to database	Done	Admins add, edit and delete events, cancel bookings, change roles and download reports; each change appears in MySQL and in audit_logs.

1. Automated test results

Running npm test in the backend folder sends real requests to the running API and database and reports 47 passed, 0 failed (run on 23 Oct 2026).

Area	Checks	Examples	Result
Login and sessions	5	Admin logs in; wrong password gives 401; /me returns the logged-in user	Pass
Registration	3	New user registers; duplicate email gives 409; weak password rejected	Pass
Access control	4	Normal user blocked from admin API (403); anonymous blocked (401); user cannot create events; booking needs login	Pass
SQL injection	4	Injection in login email and password does not authenticate; search treats % and quotes as plain text	Pass
Event validation	4	End before start, capacity 0 and past date are rejected; capacity cannot drop below booked seats	Pass
Event add, edit, search	4	Admin creates a 5-seat event, restricts it to one school, edits it; search finds it	Pass
Booking rules	5	Reference issued; seats left falls to 4; duplicate blocked; wrong school blocked; details must match account	Pass
Concurrency	2	25 simultaneous requests for 4 seats: exactly 4 succeed; event ends exactly full	Pass
Registration card and My bookings	3	Card is a PDF; another user's card is refused; My bookings lists the booking	Pass

Reports	4	Bookings report as PDF, CSV and JSON; events summary as PDF	Pass
Audit log	1	The log records who booked	Pass
Passwords	4	Admin reset gives a temporary password that works; forgot password works with the right student number and fails with a wrong one	Pass
Cleanup rules	6	User with bookings cannot be deleted; admin sees the event's bookings, cancels one and the seat returns; deleting an event removes its bookings	Pass

2. User acceptance test cases (browser)

These flows were run end to end in a real browser (Chromium) against the running system on 23 Oct 2026 and each produced the expected result with no unexpected console errors. The only console errors came from deliberately invalid input, which correctly returned 400 or 404.

ID	Scenario	Steps	Expected result	Result
T01	Guest browses events	Open the home page	Tickets show date, venue, seats left	Pass
T02	Register with empty form	Choose Create account with nothing filled	Messages list every missing field; nothing saved	Pass
T03	Login with wrong password	Enter a wrong password	One general error message, no login	Pass
T04	Student login	Log in as the demo student	Events page with the student's name in the top bar	Pass
T05	Search events	Type "career"	Only matching events remain	Pass

T06	Book a seat	Open Entrepreneurship Talk, confirm booking	Reference CS-2026-00032 shown; card on My bookings	Pass
T07	Restricted event	Open Moot Court Finals as a computing student	Message that the event is for another school	Pass
T08	Full event	Open Open Mic Night (3 of 3 booked)	Message that no seats remain	Pass
T09	Download card	Choose Download PDF	PDF with event, name, reference and QR code	Pass
T10	Help and dark mode	Open Help, switch to dark mode	Page readable; choice remembered	Pass
T11	Mobile layout	Open the site at 390 px wide	No sideways scrolling	Pass
T12	Admin login	Log in as admin	Redirected to the dashboard	Pass
T13	Event form validation	New event, save with empty form	Field messages, nothing saved	Pass
T14	Delete event	Choose Delete on an event	Confirmation window before anything is removed	Pass
T15	Bookings, users, reports, audit	Open each page	Tables fill from the database	Pass
T16	Report PDF	Download the bookings report	PDF with logo, CampusSeat name, date, total, page numbers	Pass
T17	Logout	Choose Log out	Back to the guest view	Pass

3. Security and data integrity

Marking guide item	How CampusSeat meets it	Verified by
Password hashing	bcrypt hashes; plain passwords are never stored	Code review of auth routes
Prepared statements	Every query uses placeholders (mysql2 execute)	SQL injection tests, section 2
Access levels	user and admin roles checked on the server for every admin route	Access control tests, section 2
Login, logout, register, forgot password	All four work; forgot password needs email plus student number	Tests and T02 to T04, T17
Audit logs	Logins, failed logins, bookings, cancellations, event and user changes, report downloads	Audit test and admin audit page
No double booking	One database transaction locks the event row, then checks date, duplicate, school and capacity	25 simultaneous requests for 4 seats gave exactly 4 bookings
Brute-force slowing	Login and registration are rate limited	Code review (limit raised in tests only)
Browser protections	Helmet security headers with a content security policy; no inline scripts	Page loads with the policy on
Input validation	Server checks every field; the form shows the message beside the field	T02, T13 and validation tests
Database design	Six tables with primary and foreign keys, indexes and cascade rules (normalised to 3NF)	database/schema.sql

4. Known limitations and deployment

- Forgot password checks email plus student number instead of sending an emailed link, because no mail server is set up. Say this openly in the demo.
- Optional two-factor login is not built.
- The class diagram in the documentation differs slightly from the final schema (for example, seats remaining is calculated, not stored). Update the diagram before submission or explain the difference.
- Seed events are dated a few weeks ahead of 23 Oct 2026; run npm run seed again if they pass, because past events cannot be created.
- Not tested: heavy load with many hundreds of users, and very old browsers.

Deployment. The marking guide accepts an installer or a live hosted site. The simplest route is one Node host (for example Render or Railway) plus a managed MySQL database: set the same environment values as .env (database host, user, password, a new JWT_SECRET, NODE_ENV=production), run the schema file once, run npm run seed once, build the frontend, and start the backend. The backend serves the built site, so there is one address to share. If hosting is not possible, demonstrate from the laptop with the Windows steps in the User Manual, section 1.